

EEP 596A Computer Vision: Classical and Deep Methods

Homework 7 Report

Po Peng

2025-11-17

Task 1 — Stereo Rectification Check

I loaded tsukuba_left.png and tsukuba_right.png in grayscale using:

```
tb_left = cv.imread("tsukuba_left.png", cv.IMREAD_GRAYSCALE)
tb_right = cv.imread("tsukuba_right.png", cv.IMREAD_GRAYSCALE)
```

Then I horizontally stacked the two images to visually check whether they are rectified.

Are the Tsukuba images rectified?

Yes.

After stacking the left and right images side-by-side, I checked the alignment of corresponding features.

I observed that edges and object boundaries appear on the same row in both images without noticeable vertical displacement.

Since corresponding pixels have the same y-coordinate, the disparity changes only along the x-direction, which means the stereo pair is rectified.

Rectified stereo pair (left | right)



Task 2 — Scanline Matching (SAD)

I extracted row **152** from both the left and right stereo images, using columns **102–202** (100 pixels). For each disparity value d , I shifted the right scanline by d pixels and computed the Sum of Absolute Differences (SAD):

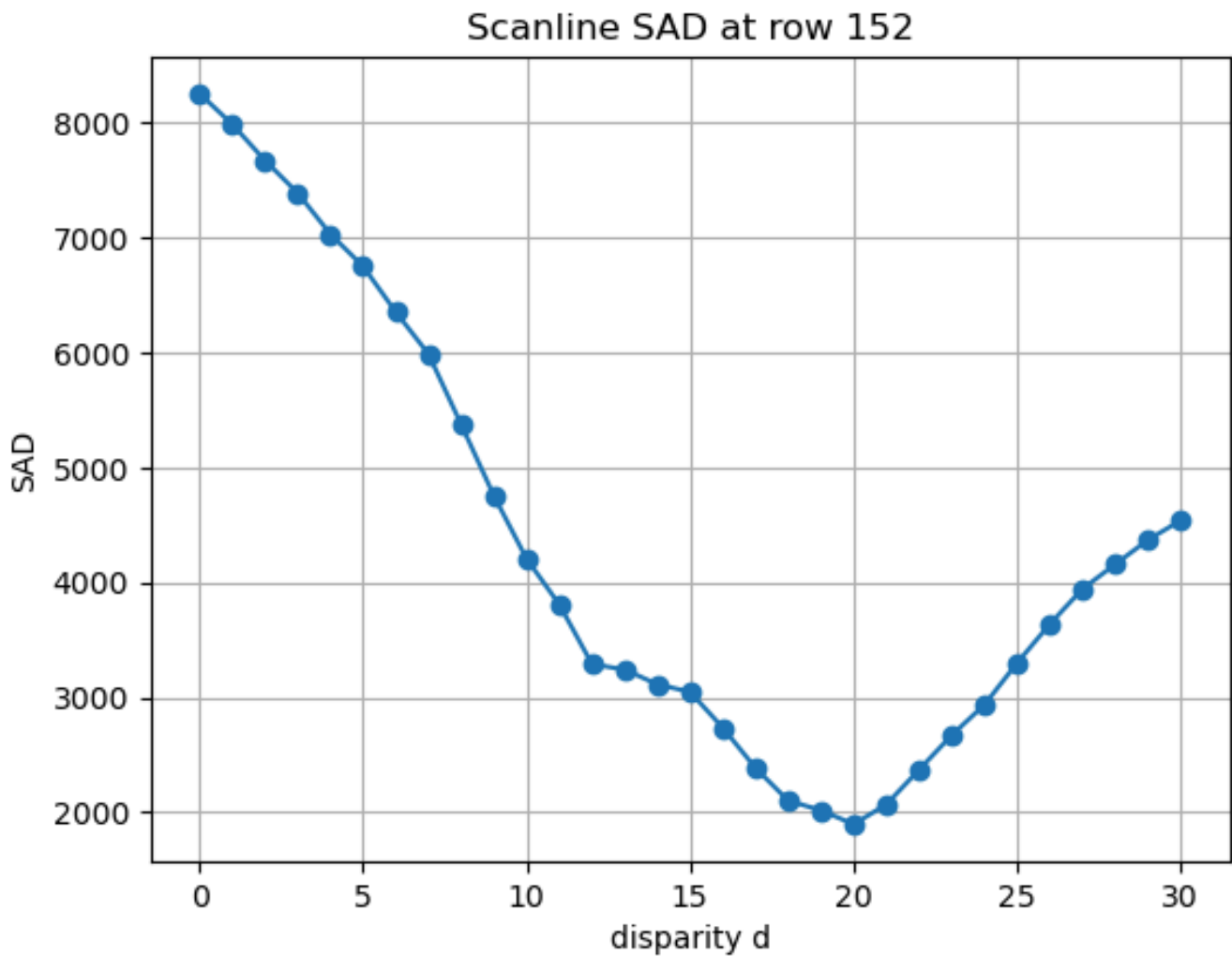
$$g(d) = \sum_x (|I_{L(x)} - I_{R(x-d)}|)$$

This measures how well the two scanlines match at each disparity. A smaller SAD value indicates a better alignment between the left and right patches.

After evaluating disparities from $d = 0$ to $d = 30$, the minimum error occurs at:

Best disparity: $d = 21$

This result is consistent with the expected scene geometry of the Tsukuba dataset, where closer objects exhibit larger disparities.



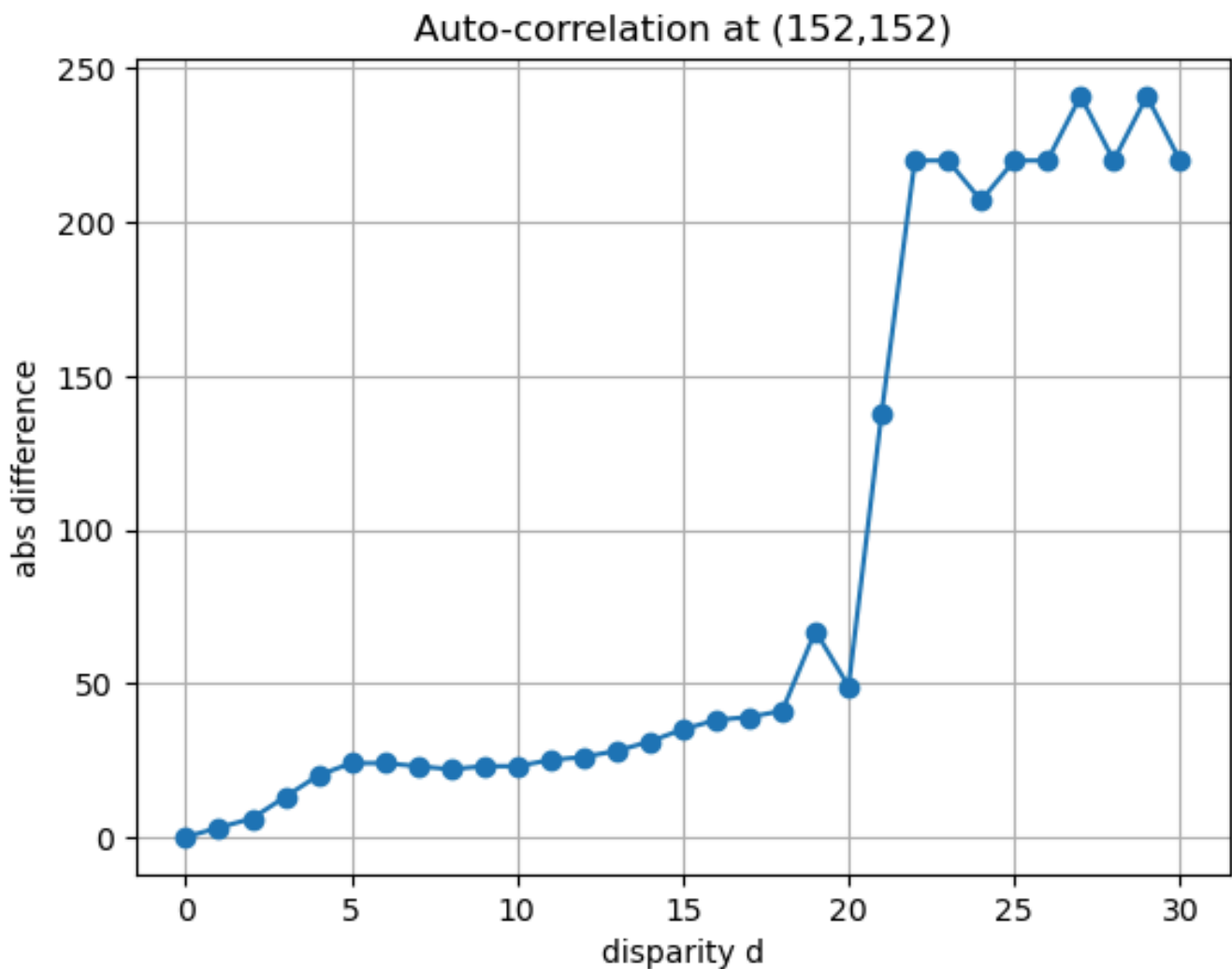
Task 3 — Auto-Correlation (Unsmoothed)

To compute the auto-correlation, I shifted the right image horizontally by disparities $d = 0 \dots 30$. For each shift, I computed the pixel-wise absolute difference between the original right image and the shifted version:

$$A(d) = |I(x, y) - I(x - d, y)|$$

I then selected pixel (152, 152) and recorded its absolute-difference value for every disparity d . This produces a 1D auto-correlation curve that shows how much the pixel differs from its shifted counterparts.

The resulting curve is shown below.



Task 4 — Smoothed Auto-Correlation

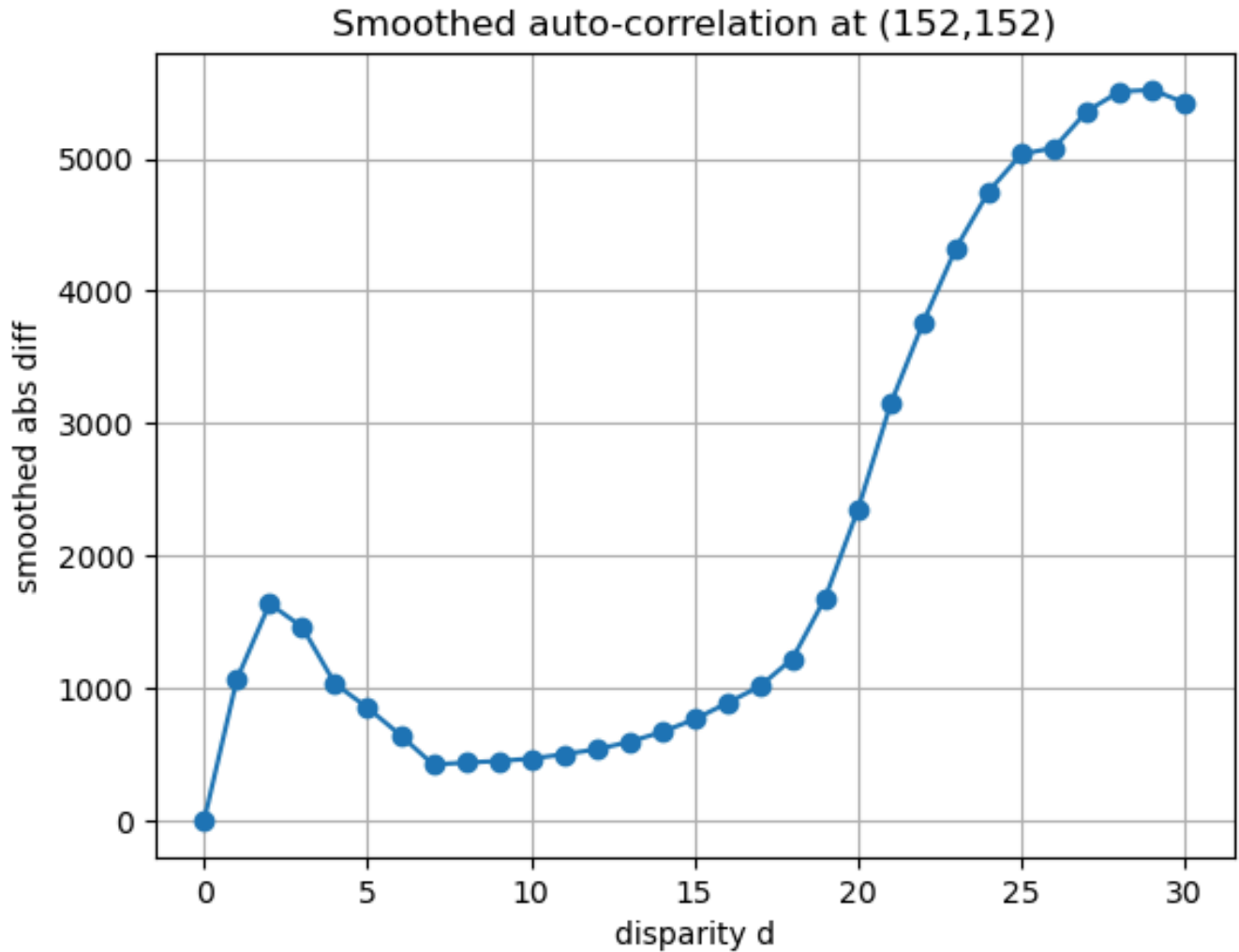
To reduce noise in the auto-correlation curve, I applied a 5×5 box filter (all ones, without normalization) to each absolute-difference image computed in Task 3.

The smoothed auto-correlation is obtained by

$$A_s = A * K_{\{5 \times 5\}}$$

where $K_{\{5 \times 5\}}$ is a 5×5 kernel of ones.

After smoothing, I again evaluated the value at pixel (152, 152) for every disparity d . The resulting 1D curve is much smoother and reveals the same overall trend with reduced local fluctuations.



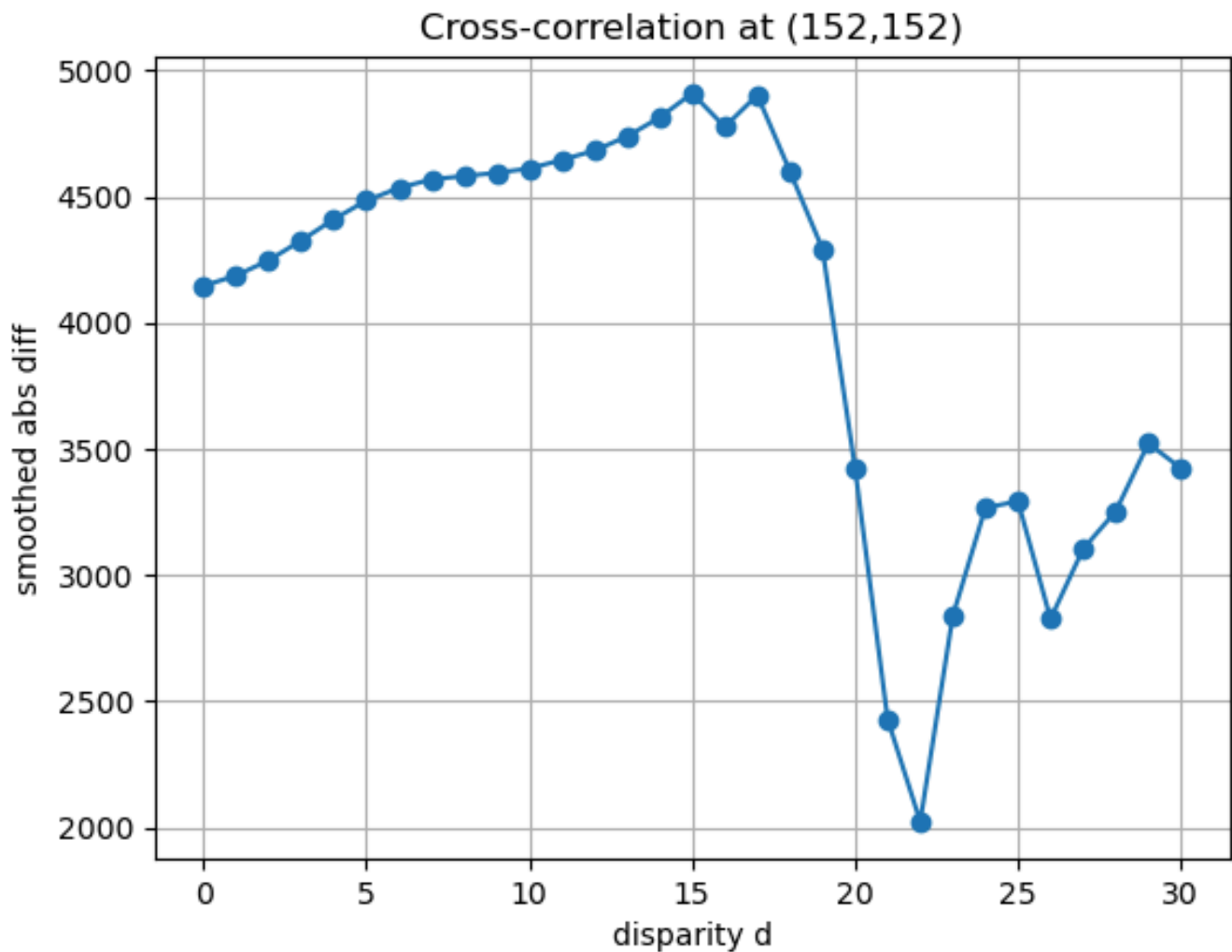
Task 5 — Cross-Correlation

In this task, instead of comparing shifted versions of the right image with the right image itself, I compared each shifted right image with the **left** image. For each disparity value $d = 0 \dots 30$, I computed the smoothed pixel-wise absolute difference:

$$C(d) = |I_{L(x,y)} - I_{R(x-d,y)}|$$

As in the previous tasks, I selected pixel (152, 152) to form a 1D cross-correlation curve across all disparity values.

The resulting smoothed cross-correlation curve is shown below.



Task 6 — Disparity Map (Left → Right)

Using the smoothed cross-correlation cost volume from Task 5, I computed the left-to-right disparity map by selecting, for each pixel, the disparity value that minimizes the matching cost:

$$d(x, y) = \arg \min_d C_s(x, y, d)$$

where C_s is the 5×5 smoothed absolute-difference cost. The right image was shifted by $d = 0 \dots 30$ pixels, and the cost for each disparity was stored in a 3D cost volume.

The resulting disparity map is shown below. As expected, textured regions of the scene produce stable disparity values, while low-texture or occluded regions exhibit noise due to the pixel-wise matching approach.

Disparity L→R



Task 7 — Disparity Map (Right → Left)

To compute the right-to-left disparity map, I reversed the matching direction. Instead of shifting the right image as in Task 6, I shifted the **left** image by $d = 0 \dots 30$ pixels and compared it with the right image. For each pixel, I constructed a smoothed absolute-difference cost volume and selected the disparity that minimized the cost:

$$d_{R(x,y)} = - \arg \min_d C_{s(x,y,d)}$$

The negative sign ensures consistency with the convention that right-to-left disparities should be the opposite of left-to-right disparities.

As expected, the resulting disparity map is noisier than the left-to-right version. This is because the right image contains more occluded or unmatched pixels, which produces ambiguous cost values and unstable disparity estimates.

Disparity R→L (magnitude)



Task 8 — Left-Right Consistency Check

To ensure reliable disparity estimates, I applied a left-right consistency check. A pixel is considered valid only when the left-to-right disparity agrees with the right-to-left disparity at the corresponding matched pixel:

$$d_{L(x,y)} = -d_{R(x-d_{L(x,y)},y)}$$

If this condition fails, I marked the pixel as inconsistent and set its disparity to 0. This removes disparities in occluded or ambiguous regions where the two directions disagree.

Below is the cleaned disparity map after applying the LR consistency rule.

LR Consistency Cleaned Disparity



Task 9 — 3D Reconstruction

Using the cleaned disparity map from Task 8, I converted disparity values into depth using the standard stereo reconstruction equations:

$$Z = \frac{f * B}{d}$$

$$X = (x - c_x) * Z$$

$$Y = -(y - c_y) * Z$$

For each valid pixel with $d > 0$, I computed its 3D coordinates (X, Y, Z) and combined them with the corresponding RGB values from the left image. I then saved all valid points into a PLY point cloud file (`kermit.ply`). Each vertex in the PLY file follows the format:

`x y z r g b`

This produces a sparse but meaningful 3D reconstruction of the Tsukuba scene, where depth is inversely related to disparity.

Below is a visualization of the reconstructed depth image:

